

Same model, opposite ends of the clock

I tested TypeSafe's Jev on two jobs that have nothing in common. One reads months of texts between two people and finds the day something changed. The other sits inside a live phone call and decides whether the caller is done talking. I didn't train it on either one. I just changed the questions.

40/40

meaning-only turning points found by Jev (sentiment lexicon: 0/40), with 0 false alarms in 40 steady threads

67.0%

zero-shot derailment accuracy on Wikipedia talk pages (CRAFT, trained: 66.5%)

333 ms

less dead air per turn vs a silence timeout (Jev + lexical stacked; tuned for 5% interruptions, 6.5% on held-out calls vs 5.0%)

\$19.18

to read all 4.6M messages of Grey Mirror's signal cohort with eight typed questions per day

Contents

1. [Case study 1 · Grey Mirror x Jev: relationship intelligence at System One speed \(turning points, derailment forecasting, economics\)](#)
2. [Case study 2 · Jev in the loop: when has the caller finished speaking?](#)
3. [Reproducibility, data and limits](#)

Case study 1 · Grey Mirror × Jev: relationship intelligence at System One speed

I built Grey Mirror to read a whole relationship's message history, not a screenshot. It measures who reaches out, who replies, how fights start and whether anyone repairs them. Publishing our own research taught me three things about this problem. Real signal is rare. A good tool has to know when to stay quiet. And a "finding" only counts if it stands out from that couple's normal rhythm, not everyone else's.

That's why Jev caught my attention. It's cheap enough to read every single message, it answers in types my code can use directly, and its probabilities are calibrated, so I can decide when not to say anything. I wanted to see if that holds up on data anybody can check. So everything below is public or synthetic. I did not use a single customer's conversation, and none was sent to any model.

1A · Turning points: can Jev see a relationship change that the words hide?

Grey Mirror already publishes a turning-point benchmark: 40 threads where nothing changes and 40 where I planted one change on a known day. We score 0% false alarms, 100% detection, 100% right direction and 0 days off. Honestly, I wanted to know if that test is too easy. So I rebuilt it (120 threads, 127,751 messages, 90 days each) and added a third group it never tested.

- **Steady (40):** no change, with decoys: two or three fights that are repaired the same day, and in half the threads a "busy week" with lower volume but unchanged warmth. Any detection is a false positive.
- **Overt (40):** the partner cools or warms, and the cold register is explicitly negative ("stop asking. it was bad."), replies slow down and volume drops. Every surface signal moves.
- **Subtle (40):** the partner cools or warms *in meaning only*. The cold register is polite, deflecting or passive-aggressive ("oh great, love that for me 😊", "sounds good in theory, let's play it by ear"), and message volume, reply timing, length, question rate and sentiment vocabulary are held constant. This is the case Grey Mirror's own benchmark report lists as a known weakness (sarcasm ~65%).

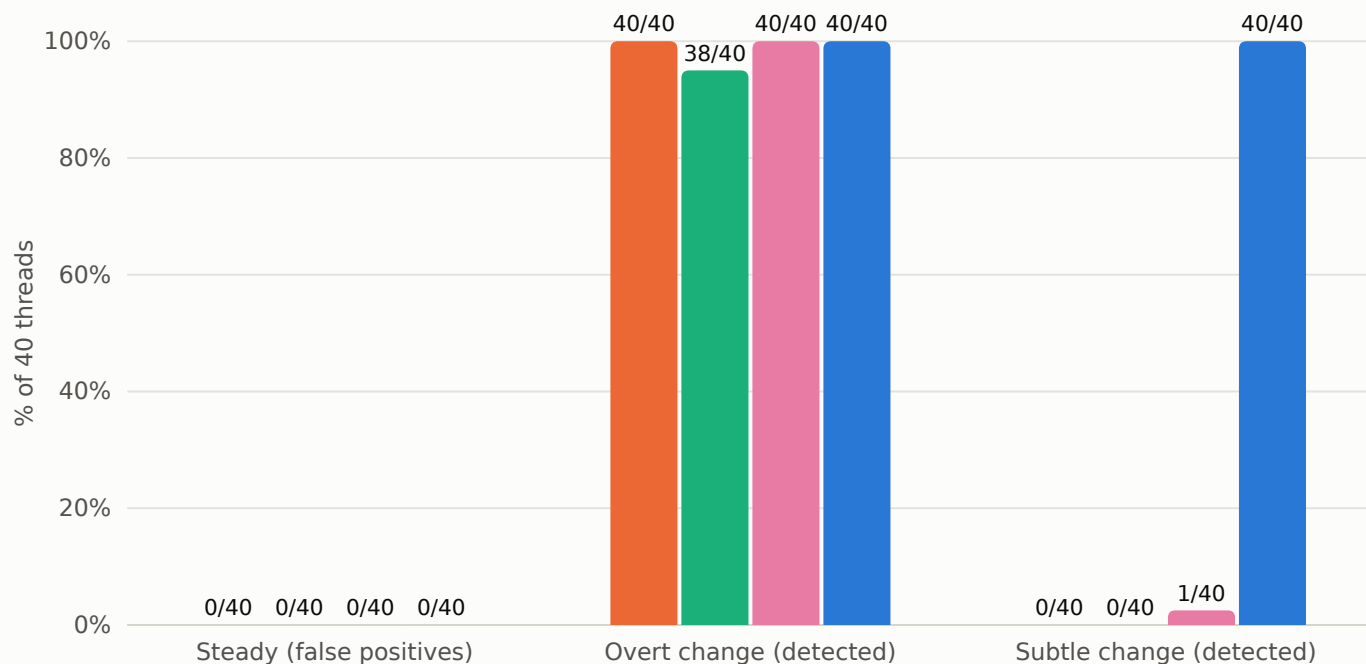
Partner surface statistic	Overt arm: warm → cold	Subtle arm: warm → cold
VADER compound (partner)	0.31 → -0.05	0.38 → 0.38
Partner messages / day	5.27 → 4.76	5.36 → 5.38
Characters / message	36.19 → 29.44	38.24 → 41.51
Share of messages with a question	0.32 → 0.18	0.34 → 0.35
Reply delay (min)	30.81 → 51.72	26.80 → 27.14

The subtle arm is meaning-only by construction: the partner's sentiment score, volume, question rate and reply delay are essentially unchanged across the planted change, while the overt arm moves on every surface statistic.

Method. For each thread-day, one Jev request carries that day's messages as `state` and eight typed questions (two Scores: warmth and engagement; six Nouns: distant-but-polite, sarcasm or passive aggression, affection, enthusiasm for plans, open conflict, repair). A pre-registered composite (warmth + engagement + affection + plans – distance – sarcasm, each z-scored within the thread) feeds one change-point detector shared by every layer: maximum two-sample t-statistic over candidate days, null tested by circular block permutation (999 permutations, 7-day blocks, $p < 0.01$, effect size ≥ 0.8 SD). The settings were fixed before any Jev output existed. Baselines go through the identical detector: VADER sentiment of the partner's messages, and four behavioral metrics (volume, length, question rate, reply delay).

Turning points flagged, by arm (same detector for every layer)

— Sentiment lexicon (VADER)
 — Behavioral metrics
 — Lexicon + behavioral
— Jev (8-question composite)



Signal layer	False positives (40 steady)	Overt: detected	Overt: direction	Overt: median error	Subtle: detected	Subtle: direction
Sentiment lexicon (VADER)	0/40	40/40	100%	0 d	0/40	—
Behavioral metrics	0/40	38/40	100%	0 d	0/40	—
Lexicon + behavioral	0/40	40/40	100%	0 d	1/40	0%
Jev (8-question composite)	0/40	40/40	100%	0 d	40/40	100%
Jev (warmth score only)	0/40	40/40	100%	0 d	40/40	100%

One subtle thread (subtle_000): signal z-scored within the thread, 7-day rolling mean

— Sentiment lexicon
 — Behavioral
 — Jev composite



What the data says

- A plain sentiment lexicon already matches Grey Mirror's published benchmark on overt changes: 40/40 detected, 100% correct direction, 0/40 false positives. The published benchmark does not separate a smart signal layer from a dumb one; the subtle arm does.
- On meaning-only changes the lexicon finds 0/40 and behavioral metrics 0/40 (direction correct in — of those): the surface statistics barely move, by construction.
- Jev, reading one day at a time through eight typed questions, flags 40/40 subtle changes (100% correct direction, median localisation error 0.0 days) and 40/40 overt ones, with 0/40 false positives in threads that contain repaired fights and busy weeks.
- Ablation: the single warmth score alone flags 40/40 subtle changes (0/40 false positives), so the fan-out questions do not add detection power.

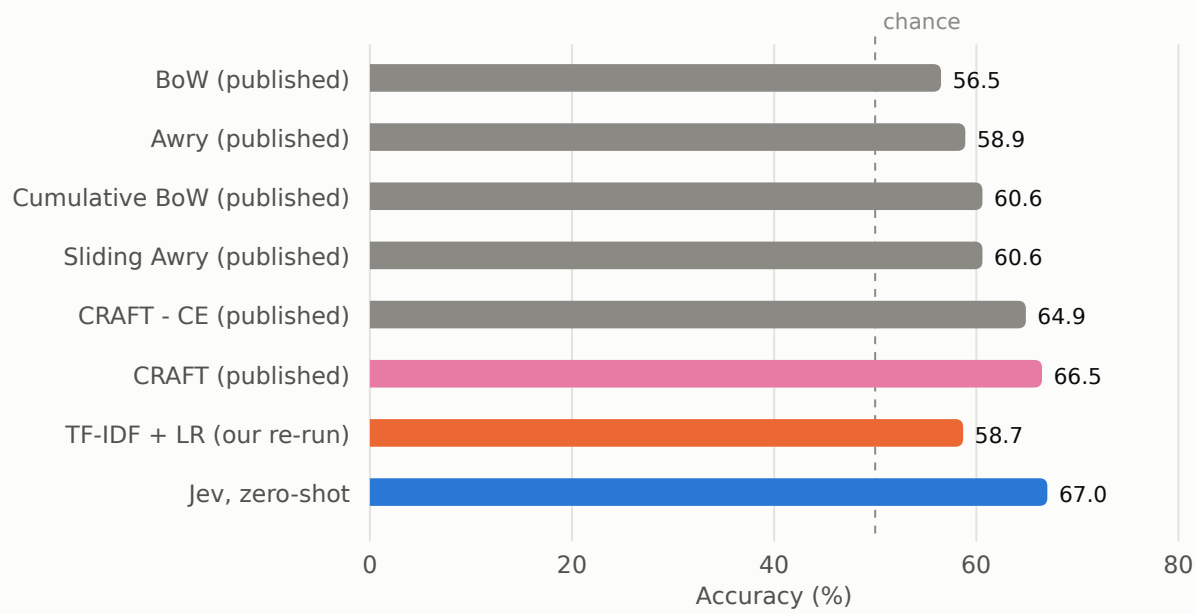
Usage: 10,800 Jev requests, 12,681,349 input tokens, \$0.5326 at list price. The 120 threads are also exported as WhatsApp-style .txt and CSV files ([data/prepared/gm_uploads/](#)) so Grey Mirror's own detector can be run on exactly the same threads for a head-to-head.

1B · Before the fight: forecasting derailment in real conversations

In our own conflict research, escalation outran repair about 36 to 1. Spotting a fight after it happens is easy. What I actually want is to see one coming. Conversations Gone Awry, a public dataset from Cornell, is the standard test for that. It pairs real conversations from the same page, where one ends in a personal attack and the other stays civil. I ran it exactly the way the best published model, CRAFT, was scored (Chang & Danescu-Niculescu-Mizil, 2019). The attack itself is never shown. Jev makes a call after every comment, and a flag only counts if it fires before the attack happens. CRAFT was pre-trained on these platforms and fine-tuned on thousands of labeled conversations. Jev got zero examples. The only thing I tuned was one cutoff number, and I tuned it on a separate validation set.

Wikipedia talk pages

Wikipedia talk pages: forecasting accuracy on the test split



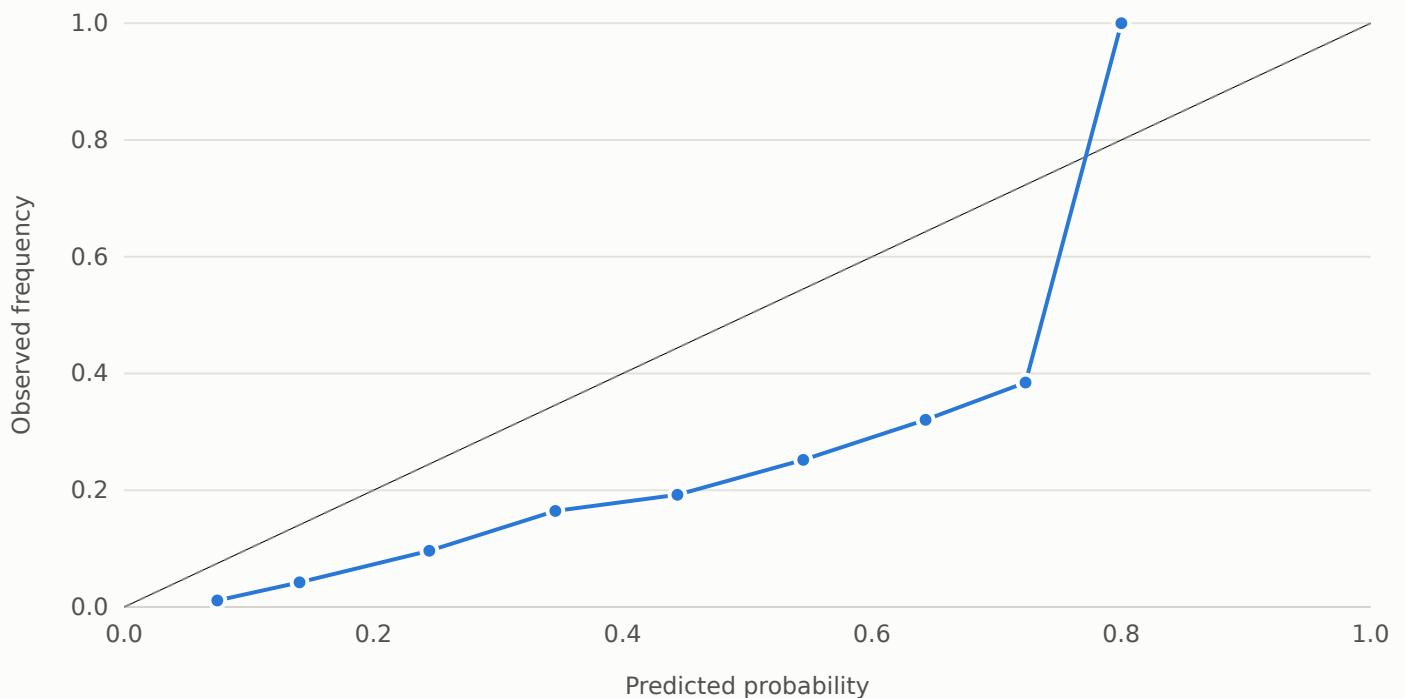
Wikipedia talk pages: test split (n = 840), CRAFT protocol

Model	Accuracy	Precision	Recall	FPR	F1	AUC	Pairwise
BoW (published, supervised)	56.5	55.6	65.5	52.4	60.1	—	—
Awry (published, supervised)	58.9	59.2	57.6	39.8	58.4	—	—
Cumulative BoW (published, supervised)	60.6	57.7	79.3	58.1	66.8	—	—

Model	Accuracy	Precision	Recall	FPR	F1	AUC	Pairwise
Sliding Awry (published, supervised)	60.6	60.2	62.4	41.2	61.3	—	—
CRAFT - CE (published, supervised)	64.9	64.4	66.7	36.9	65.5	—	—
CRAFT (published, supervised)	66.5	63.7	77.1	44.1	69.8	—	—
TF-IDF + LR (our re-run, supervised)	58.7	59.4	54.8	37.4	57.0	0.621	62.4%
Jev: 'next comment is an attack?' (zero-shot)	67.0	64.1	77.4	43.3	70.1	0.732	74.2%
Jev: 'heading for a breakdown?' (zero-shot)	67.6	65.7	73.8	38.6	69.5	0.747	76.0%
Jev: tension score (zero-shot)	67.1	65.1	73.8	39.5	69.2	0.726	72.7%

Wikipedia talk pages: calibration of every forecast ('is the NEXT comment an attack?')

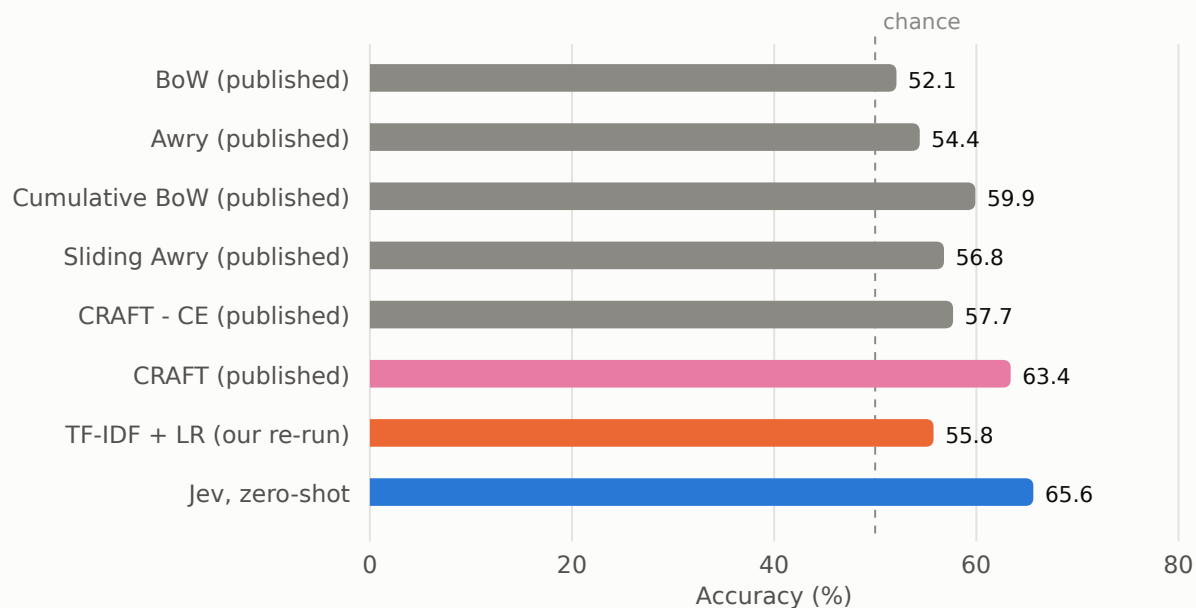
— Jev



- Zero-shot, with one threshold tuned on the validation split, Jev reaches 67.0% accuracy (95% CI 63.8–70.2) and F1 70.1, against 66.5% / 69.8 for CRAFT, a model pre-trained on this platform's conversations and fine-tuned on 2,508 labelled ones (above CRAFT by 0.5 points).
- Threshold-free: AUC 0.732 (95% CI 0.696–0.764); given a matched pair from the same page, Jev ranks the conversation that derails higher 74.2% of the time.
- Early warning: on correctly flagged derailments Jev first fires 3.8 comments before the attack on average (CRAFT: on average 3 comments before the attack).
- Against our supervised TF-IDF model on the same test conversations, Jev's AUC is +0.110 (95% CI +0.072 to +0.147).
- Calibration across all 4,365 individual forecasts: ECE 0.144, Brier 0.106, base rate 9.6%.

Reddit r/ChangeMyView

Reddit r/ChangeMyView: forecasting accuracy on the test split



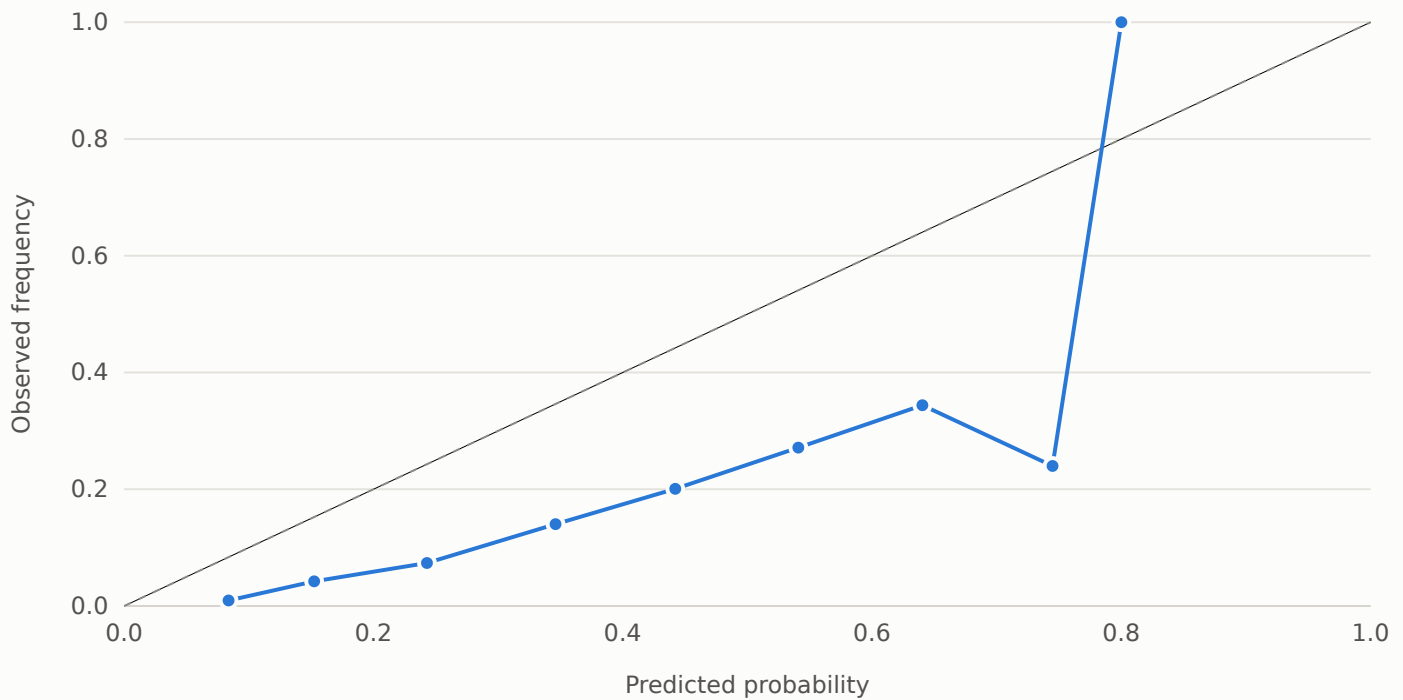
Reddit r/ChangeMyView: test split (n = 1368), CRAFT protocol

Model	Accuracy	Precision	Recall	FPR	F1	AUC	Pairwise
BoW (published, supervised)	52.1	51.8	61.3	57.0	56.1	—	—

Model	Accuracy	Precision	Recall	FPR	F1	AUC	Pairwise
Awry (published, supervised)	54.4	55.0	48.3	39.5	51.4	—	—
Cumulative BoW (published, supervised)	59.9	58.8	65.9	46.2	62.1	—	—
Sliding Awry (published, supervised)	56.8	56.6	58.2	44.6	57.4	—	—
CRAFT - CE (published, supervised)	57.7	56.1	71.2	55.7	62.8	—	—
CRAFT (published, supervised)	63.4	60.4	77.5	50.7	67.9	—	—
TF-IDF + LR (our re-run, supervised)	55.8	55.2	61.1	49.6	58.0	0.575	57.9%
Jev: 'next comment is an attack?' (zero-shot)	65.6	64.3	70.2	38.9	67.1	0.712	72.2%
Jev: 'heading for a breakdown?' (zero-shot)	65.6	64.9	67.7	36.5	66.3	0.711	73.5%
Jev: tension score (zero-shot)	66.0	65.0	69.4	37.4	67.1	0.719	73.3%

Reddit r/ChangeMyView: calibration of every forecast ('is the NEXT comment an attack?')

— Jev



- Zero-shot, with one threshold tuned on the validation split, Jev reaches 65.6% accuracy (95% CI 63.1–68.1) and F1 67.1, against 63.4% / 67.9 for CRAFT, a model pre-trained on this platform's conversations and fine-tuned on 4,106 labelled ones (above CRAFT by 2.2 points).
- Threshold-free: AUC 0.712 (95% CI 0.687–0.739); given a matched pair from the same page, Jev ranks the conversation that derails higher 72.2% of the time.
- Early warning: on correctly flagged derailments Jev first fires 3.7 comments before the attack on average (CRAFT: on average 4 comments).
- Against our supervised TF-IDF model on the same test conversations, Jev's AUC is +0.138 (95% CI +0.103 to +0.173).
- Calibration across all 7,098 individual forecasts: ECE 0.166, Brier 0.112, base rate 9.6%.

1C · What it costs to read everything

The headline from our research was simple: 97.4% of 4.6 million messages carried no emotional signal at all. If the signal is that rare, the expensive part of the job is reading everything just to find the few messages that matter. That's exactly the kind of work Jev is priced for: \$0.042 per million tokens in, nothing for what comes out.

Cost to read every message with the 1A question set (99 input tokens per message measured)

Grey Mirror corpus (published size)	Messages	Jev, 8 questions per day-window	Claude Haiku 4.5 (est.)	Claude Sonnet 5 (est.)
Signal cohort: 29 histories	4,600,611	\$19.18	\$623.98	\$1,247.96
Conflict cohort: 44 histories	2,994,932	\$12.49	\$406.20	\$812.40
Operational rollup: 347 conversations	20,794,571	\$86.70	\$2,820.36	\$5,640.73

LLM estimates assume the same input tokens plus 80 output tokens per window at public gateway list prices (retrieved 2026-09-27); they exclude the longer prompt an LLM needs to return eight structured answers.

Case study 2 · Jev in the loop: when has the caller finished speaking?

This one has nothing to do with relationships, and that's the point. Every voice AI, including the phone-call feature I run on JustLayMe, has to make the same call dozens of times a minute: the caller went quiet, so are they done or just thinking? Jump in too early and you talk over them. Wait too long and every turn ends in dead air. Most voice agents still just wait for a set amount of silence. This is a snap judgment made in real time, which is exactly what TypeSafe says Jev was built for. Nobody trained it for this.

Data. Switchboard: two-person telephone calls with manually corrected word-level forced alignments (Mississippi State ISIP release). Every clean pause (no overlap, the other side silent) is labelled by what actually happened next: the same speaker resumed (a *hold*, where responding would have been an interruption) or the other person took the floor (a *shift*). Pauses followed only by a backchannel ("uh-huh") are excluded. Text is lowercase with no punctuation, like streaming speech recognition, so no annotator punctuation can leak the answer. The held-out evaluation sample is 2,400 pauses from 252 calls never used for training or tuning; mid-turn pauses have a median of 456 ms (90th percentile 980 ms). We first tried the AMI meeting corpus and rejected it for this purpose: its word

times are contiguous within transcription segments, so short mid-turn pauses are invisible.

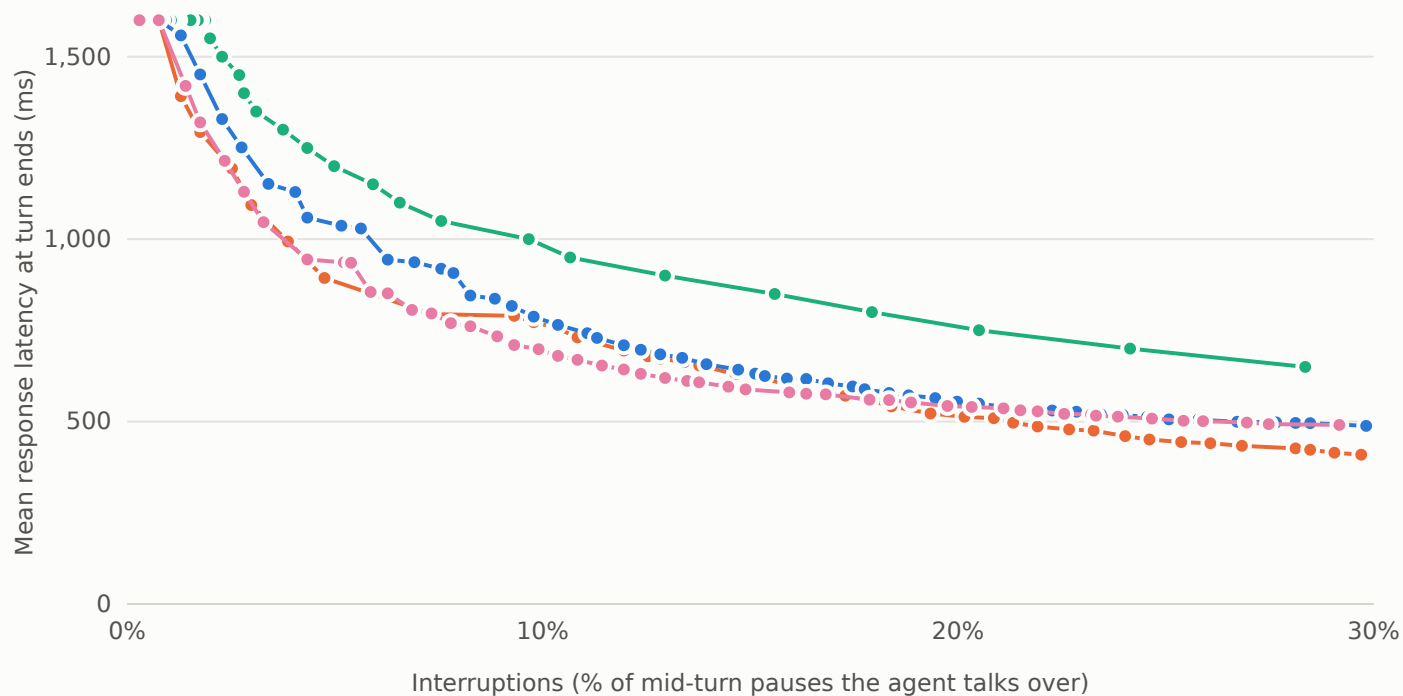
Method. At each pause Jev receives the last ~45 seconds of transcript and four questions in one request: a Noul "has the speaker finished and handed over the turn?", Noul for "stopped mid-phrase" and "just asked a question", and a five-level yield Score. Requests are sent one at a time so every latency is a clean round trip, and the simulation charges each decision its own measured latency. The agent confirms silence at 200 ms, asks the model, and then either answers as soon as the model returns or waits for a fallback timeout. Two policy families are searched: a gate (answer now if $p \geq \theta$, else wait T) and an adaptive timeout (wait $\max(\text{model latency}, a + b \cdot (1 - p))$). Operating points are chosen on a separate dev sample of 1,200 pauses and reported on eval. The baselines are a silence timeout and a supervised logistic regression over lexical features, trained on 80,000 in-domain pauses.

Is the caller done? 2,400 held-out pauses, 28.9% are real turn ends

Model	AUC (95% CI)	ECE	Brier	AUPR
Lexical model (supervised, 80k examples)	0.832 (0.814-0.848)	0.018	0.142	0.689
Jev 'done?' (zero-shot)	0.848 (0.832-0.864)	0.065	0.144	0.648
Jev yield score (zero-shot)	0.864 (0.848-0.878)	0.076	0.138	0.686
Jev 4-question fan-out + 5-weight LR	0.864 (0.848-0.879)	0.038	0.134	0.692
Jev + lexical stacked	0.875 (0.860-0.889)	0.037	0.127	0.718

The endpointing trade-off: interruptions vs response latency (lower-left is better)

— Silence timeout — Lexical (supervised) — Jev (zero-shot) — Jev + lexical

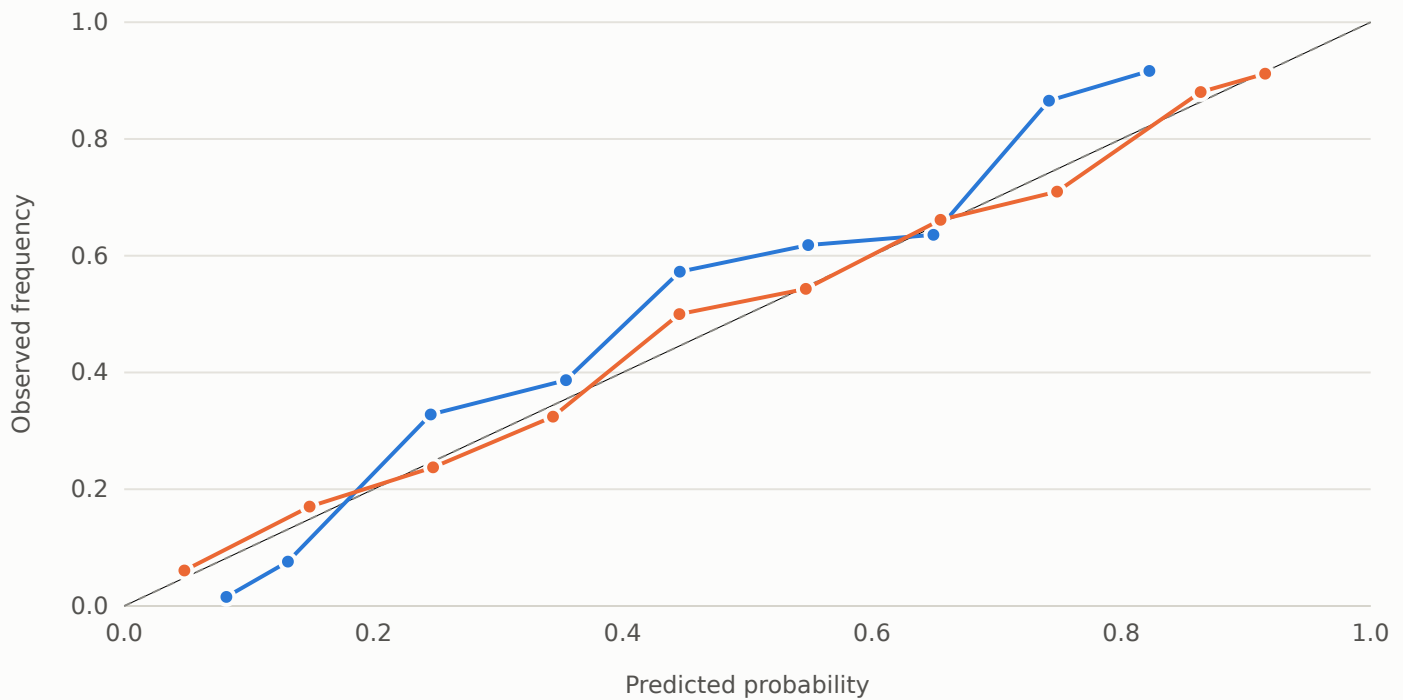


Mean response latency at turn ends, operating point chosen on dev, reported on eval

Endpointing policy	Ceiling 5% interruptions	Ceiling 10%	Ceiling 20%
Silence timeout only	1,200 ms (5.0% interrupts)	1,000 ms (9.7% interrupts)	800 ms (17.9% interrupts)
Lexical model (supervised, 80k examples)	993 ms (3.9% interrupts)	795 ms (9.5% interrupts)	522 ms (19.3% interrupts)
Jev 'done?' (zero-shot)	952 ms (5.7% interrupts)	675 ms (13.4% interrupts)	518 ms (25.4% interrupts)
Jev 4-question fan-out + 5-weight LR	925 ms (6.6% interrupts)	671 ms (12.3% interrupts)	524 ms (23.1% interrupts)
Jev + lexical stacked	867 ms (6.5% interrupts)	616 ms (13.9% interrupts)	514 ms (23.8% interrupts)

Calibration: predicted vs observed turn ends (eval)

— Jev 'done?' — Lexical (supervised)



What the data says

- The problem, measured: a silence-only agent needs a 1.20 s timeout to keep interruptions near 5% (5.0% on eval), so every turn end costs the caller 1,200 ms of dead air. Mid-turn pauses in this data have a median of 456 ms and a 95th percentile of 1,196 ms.
- A supervised lexical model trained on 80,000 in-domain pauses reaches AUC 0.832: the words before a pause carry most of the signal, which is why a text-only model is worth putting in the loop at all.
- Jev, zero-shot with one question, reaches AUC 0.848 and ECE 0.065; the four-question fan-out with five fitted weights reaches 0.864, and stacking Jev with the lexical model 0.875.
- Jev 'done?' vs the supervised lexical model: AUC difference +0.016 (95% CI -0.000 to +0.033).
- Measured round-trip latency from a cloud container through Vercel AI Gateway, one request at a time: median 258 ms, p90 314 ms, p99 430 ms. Each simulated decision uses its own request's measured latency.
- At a 5% interruption ceiling (chosen on dev), Jev + lexical stacked answers turn ends in 867 ms on average (6.5% interruptions on eval), saving 333 ms per turn against the silence timeout.

Reproducibility, data and limits

How we built with Jev

- **Arithmetic stays in code.** Jev never counts messages, compares dates or computes a trend. It answers one semantic question per day-window; the change-point statistics, thresholds and policy timing are ordinary code (TypeSafe's jaggedness guide, items 2-3).
- **Small states.** One day of messages, one conversation prefix, or the last 45 seconds of a call per request, never a whole history, to avoid the context-rot failure mode the docs warn about.
- **Literal questions with criteria.** Every Noul spells out what yes and no mean (for example, 'polite but closed replies' counts as distant) because Jev answers the question as written.
- **Speculative fan-out.** All questions about one state go in one request: eight in 1A, three in 1B, four in 2. They share one latency and one state ingestion.
- **Calibration used as a control signal.** Probabilities feed thresholds and timeout policies directly; the only fitted parameters are thresholds or a five-weight logistic layer, always fit on a dev split and reported on held-out data.
- **Pre-registration.** Detector settings, composites and question wording were fixed before any Jev output was seen, and baselines run through the identical detector.

The requests

- ▶ 1A · one thread-day, eight questions
- ▶ 1B · one conversation prefix, three questions
- ▶ 2 · one pause, four questions

Run it

```
cd jev-case-studies
export AI_GATEWAY_API_KEY=... # Vercel AI Gateway key; Jev is typesafe-ai/jev
./run_all.sh # downloads data, runs all experiments, rebuilds
```

Every Jev and LLM response is cached on disk keyed by a hash of the exact request, so re-running never re-spends and every number here can be replayed. Jev is called through Vercel AI Gateway's TypeSafe-compatible endpoint (`POST https://ai-gateway.vercel.sh/typesafe/v1/systemone`), model (`typesafe-ai/jev`); setting

`JEV_PROVIDER=typesafe` calls TypeSafe directly instead. Jev spend recorded for this build: \$1.57.

Data

- **Synthetic relationship threads** (1A): generated by `cs1_grey_mirror/generate_threads.py` from hand-written message banks, seeded and fully reproducible; exported in Grey Mirror's upload formats.
- **Conversations Gone Awry** (1B): Zhang et al. 2018 and Chang & Danescu-Niculescu-Mizil 2019, distributed with Cornell ConvoKit. Published CRAFT numbers are copied from that paper's Table 1.
- **Switchboard** (2): Godfrey et al. 1992; word alignments and transcripts from the Mississippi State ISIP "MS98" release. Raw transcripts are not redistributed in this repository.
- **Grey Mirror figures**: the public research package at justlay.me/research/the-signal-is-rare (aggregate data only).

Limits

- 1A uses synthetic threads with planted ground truth. It isolates one capability (reading meaning that surface statistics miss) and says nothing directly about accuracy on real relationships. Real histories are also far sparser in signal than these threads.
- 1B's datasets are balanced by construction (half the conversations derail), so absolute precision and calibration differ from the wild, where attacks are rare.
- 2 is text-only. Production endpointing also uses prosody and acoustics, which Jev cannot see; the fair reading is that Jev supplies the semantic half of the decision at a latency the loop can afford. Latency was measured from one cloud region through a gateway and will vary by deployment.
- Each experiment is a single run of a deterministic pipeline; confidence intervals come from bootstrap resampling of the evaluation items, not repeated model sampling.

Built by `report/build_reports.py` from `results/*.json`. Jev model version as reported by the API: typesafe/jev-1.13-20260917.